

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ingeniería

Ingeniería en Computación



MANITAS MX

Traductor de Lengua de Señas Mexicana con Inteligencia Artificial

Asignatura	Cómputo Móvil
Grupo	03
Semestre	2026-2
Fecha de entrega	29 de mayo de 2026
Profesor	Ing. Marduk Pérez de Lara Domínguez
Equipo	CODE&COFFEE
Integrantes	Cruz Buenavista Lesliee Sarahí Martínez Bravo Daniela Pelaez Semprun Diego Eduardo

Reporte y Trabajo de Investigación

Trabajo final

Índice

1. Requerimientos de la app.....	4
1.1. Reglas de negocio.....	4
1.2. Requerimientos funcionales.....	4
1.3. Requerimientos no funcionales.....	5
2. Viabilidad técnica.....	6
2.1. Funcionalidad propuesta.....	6
2.2. Justificación de viabilidad.....	6
3. Alcance de la app.....	6
3.1. Alcance general.....	6
3.2. MVP (producto mínimo viable).....	7
4. Wireframes de la app.....	7
4.1 Pantalla de permisos.....	7
4.2 Pantalla de bienvenida.....	8
4.3 Pantalla de detección.....	8
4.4 Pantalla de aprendizaje.....	9
4.5 Pantalla de detalle de letra.....	9
4.6 Pantalla de historial.....	10
4.7 Pantalla de privacidad.....	10
4.8 Pantalla de detección no identificada.....	11
4.9 Pantalla de permisos IA.....	11
4.10 Pantalla de detección analizando letra.....	12
5. Flujo de navegación.....	12
6. Flujo de datos y cambios de estado.....	13
6.1. Flujo de datos.....	13
6.2. Cambios de estado.....	13
7. Dispositivos y sensores.....	15
7.1. Dispositivos, tamaños y orientaciones de pantalla.....	15
7.2. Sensores.....	15
8. Demo.....	16
9. Lenguajes, herramientas y frameworks.....	17
9.1. Lenguajes de programación.....	17
9.2. Herramientas.....	18
9.3. Frameworks.....	18
10. Arquitectura y metodología.....	18
10.1. Arquitectura del proyecto.....	18
10.2. Patrones de diseño.....	19

10.3. Metodología de desarrollo.....	19
10.4. Uso de inteligencia artificial.....	19
11. Administración del proyecto.....	20
11.1. Organización del proyecto.....	20
11.2. Herramientas de administración.....	20
11.3. Equipo de trabajo y roles.....	20
12. Permisos y publicación de la app.....	21
12.1. Permisos requeridos.....	21
12.2. Adecuaciones necesarias para la publicación.....	21
13. Costos y tiempo de desarrollo.....	21
13.1. Tiempo estimado de desarrollo.....	21
13.2. Costos de desarrollo.....	22
13.3. Costos de implementación.....	22
13.4. Costos de mantenimiento.....	23
14. Referencias.....	23

1. Requerimientos de la app

Manitas MX es una aplicación móvil para iPhone y iPad orientada al aprendizaje y la práctica del abecedario dactilológico de la Lengua de Señas Mexicana (LSM). La aplicación combina captura de cámara, análisis de postura de mano mediante Apple Vision Framework y clasificación semántica mediante un modelo de inteligencia artificial.

1.1. Reglas de negocio

- La app únicamente reconoce letras individuales del abecedario dactilológico de la Lengua de Señas Mexicana, incluyendo la letra Ñ.
- En el MVP, la app no reconoce palabras, frases completas o conversaciones en LSM.
- El usuario debe conceder permiso a la app de acceder a la cámara para utilizar el módulo de detección.
- La detección únicamente sucede cuando exista al menos una mano visible dentro del área de la cámara.
- La posición de la mano debe mantenerse estable durante unos segundos para poder ser analizada.
- La app no almacena imágenes ni video de forma permanente, únicamente utiliza la captura de cámara de forma temporal para realizar el análisis de la seña.
- El historial únicamente mostrará resultados de detección, hora de detección y nivel de confianza de la detección.
- El usuario podrá eliminar registros individuales o todo el registro del historial de detecciones.
- Si la confianza en una detección es muy baja o no hay una mano visible la app muestra un estado “no identificada”.

1.2. Requerimientos funcionales

- Pantalla inicial con tutoría de uso y explicación del propósito de la app.
- Solicitar permiso al usuario para utilizar la cámara previo a iniciar la detección para el reconocimiento de letras.
- Activar cámara frontal o trasera para el módulo de captura.

- Permitir al usuario cambiar entre la cámara frontal o la cámara trasera.
- Detectar la postura de la mano mediante Apple Vision Framework.
- Extraer una descripción espacial de la posición de la mano, incluyendo la posición de los dedos (extendidos o doblados) y orientación.
- Enviar al modelo de Gemini la información necesaria para clasificar la letra detectada, incluyendo la descripción de la posición de la mano y los dedos y una imagen temporal de apoyo.
- Mostrar en pantalla la letra detectada, su equivalencia dentro del abecedario LSM y nivel de confianza de la clasificación.
- Registrar las letras detectadas e identificadas en el historial.
- Permitir la consulta del abecedario LSM con imagen, descripción y consejos para ejecutarla.
- Permitir borrar registros individuales del historial mediante un gesto de deslizamiento.
- Permitir borrar todo el historial mediante un botón.

1.3. Requerimientos no funcionales

- Usabilidad: interfaz con diseño sencillo, intuitivo y en español.
- Accesibilidad: tipografía clara, contraste visual adecuado y elementos visuales claros
- Privacidad: no debe almacenar imágenes ni video.
- Seguridad: las API keys no deben permanecer expuestas en el código fuente.
- Rendimiento: debe responder en pocos segundos y la navegación debe funcionar de manera fluida.
- Compatibilidad: debe funcionar de manera adecuada para dispositivos iPhone y iPad con iOS o iPadOS 16 o superior.
- Mantenimiento: el código debe organizarse en módulos para facilitar futuras actualizaciones.

2. Viabilidad técnica

2.1. Funcionalidad propuesta

La funcionalidad propuesta para analizar es la funcionalidad técnica principal de la app, el reconocimiento de letras del abecedario dactilológico de la Lengua de Señas Mexicano, mediante el uso de la cámara, visión artificial e inteligencia artificial generativa, capturando la mano del usuario, analizando su postura y posteriormente utilizando el modelo de inteligencia artificial Gemini 2.5 Flash para interpretar la configuración de los dedos y clasificarla con la letra correspondiente.

2.2. Justificación de viabilidad

De la funcionalidad propuesta es viable el reconocimiento de letras de la A a la Z, incluyendo la Ñ, una de los aspectos no viables de la funcionalidad para el MVP es el reconocimiento de palabras o frases completas ya que requiere un análisis de tiempo, un dataset más complejo y extenso, otro aspecto no viable es el uso completamente offline de la funcionalidad ya que requeriría una implementación de un modelo de inteligencia artificial de forma local.

3. Alcance de la app

3.1. Alcance general

Aplicación para iPhone y iPad con funcionalidad de enseñanza y traducción del abecedario dactilológico LSM con tres módulos principales, detección en vivo, aprendizaje y consulta del alfabeto, e historial de traducciones controlado por el usuario, uso de IA para clasificación de letras, interfaz en español.

Quedan fuera del alcance inicial el reconocimiento de palabras completas, frases, conversaciones completas, cuentas de usuario, versión para otros dispositivos y funcionamiento offline.

3.2. MVP (producto mínimo viable)

Para el MVP de la app se cuenta con la implementación de los módulos de cámara, de enseñanza y el historial, a su vez, se cuenta con requerimientos de privacidad y permisos por parte del usuario junto con la conexión mediante API a Gemini.

4. Wireframes de la app

A continuación se presentan los wireframes principales de Manitas MX, organizados en el orden real del flujo de navegación de la aplicación. Esta sección también incluye la documentación por pantalla, donde se especifica la funcionalidad, la información mostrada, el tipo de datos, la vigencia, el origen de los datos y las operaciones que se esperan sobre ellos.

4.1 Pantalla de permisos

Esta es la primera pantalla del flujo. Solicita autorización para usar la cámara, ya que el módulo de detección depende de este sensor para capturar la mano del usuario.

<i>Elemento</i>	<i>Descripción</i>
<i>Funcionalidad</i>	Solicitar acceso a la cámara del dispositivo.
<i>Información</i>	Mensaje de permiso y botones de permitir o no permitir.
<i>Tipo de datos</i>	Bool, String.
<i>Vigencia del dato</i>	Temporal, se define en el momento de la autorización.
<i>Origen</i>	Sistema operativo iOS y decisión del usuario.
<i>Operaciones</i>	Consulta y registro del permiso.



Imagen 4.1.1. Wireframe de pantalla de permisos

4.2 Pantalla de bienvenida

Presenta el nombre de la aplicación, su objetivo general y un botón para comenzar. Su función es orientar al usuario antes de entrar al flujo principal.

Elemento	Descripción
Funcionalidad	Introducir al usuario a la app y dar acceso al flujo principal.
Información	Nombre de la aplicación, breve texto descriptivo y botón “Comenzar”.
Tipo de datos	String.
Vigencia del dato	Estático, mientras la app esté instalada.
Origen	Bundle local de la aplicación.
Operaciones	Consulta.



Imagen 4.2.1. Wireframe de pantalla de bienvenida

4.3 Pantalla de detección

Muestra la cámara activa y el área donde el usuario coloca la mano para iniciar la traducción de señas.

Elemento	Descripción
Funcionalidad	Capturar la imagen de la mano para análisis en tiempo real.
Información	Vista de cámara, marco de detección y letra resultante.
Tipo de datos	CVPixelBuffer, Bool, String.
Vigencia del dato	Temporal, por frame capturado.
Origen	Cámara del dispositivo y procesamiento local.
Operaciones	Consulta y actualización en tiempo real.

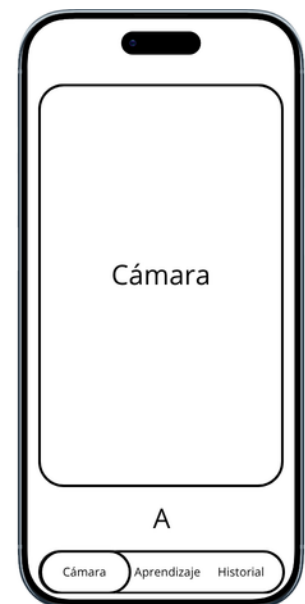


Imagen 4.3.1. Wireframe de pantalla de detección de seña

4.4 Pantalla de aprendizaje

Permite consultar el abecedario de la Lengua de Señas Mexicana en una lista ordenada.

Elemento	Descripción
Funcionalidad	Consultar letras del abecedario LSM.
Información	Lista de letras A–Z y Ñ con descripción breve.
Tipo de datos	Array de estructuras, String.
Vigencia del dato	Estática.
Origen	Datos locales integrados en la app.
Operaciones	Consulta y filtrado.

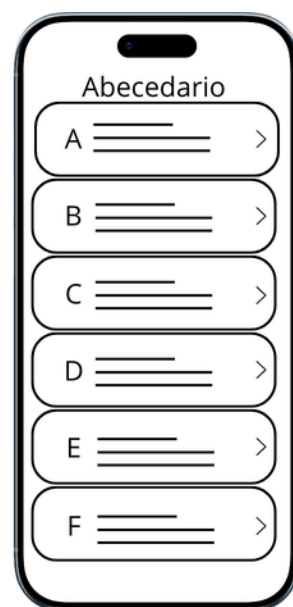


Imagen 4.4.1. Wireframe de pantalla de aprendizaje

4.5 Pantalla de detalle de letra

Muestra la explicación completa de una letra, incluyendo su forma, consejo clave y acceso a la práctica.

Elemento	Descripción
Funcionalidad	Mostrar el detalle técnico de una letra específica.
Información	Imagen de referencia, explicación, consejo y botón “Practicar”.
Tipo de datos	String, UIImage.
Vigencia del dato	Estática.
Origen	Bundle local y contenido de referencia de la app.
Operaciones	Consulta y navegación.

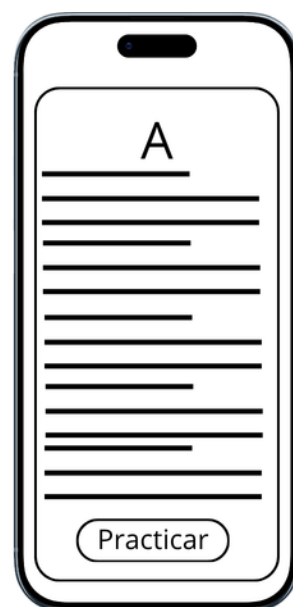


Imagen 4.5.1. Wireframe de pantalla de detalle de letra

4.6 Pantalla de historial

Muestra las letras detectadas previamente con su hora y nivel de confianza.

Elemento	Descripción
Funcionalidad	Consultar el historial de detecciones y permitir borrado individual.
Información	Letra detectada, hora, confianza y opción de borrar.
Tipo de datos	Array de registros, String, Date, Float.
Vigencia del dato	Persistente hasta que el usuario lo elimine.
Origen	Almacenamiento local del dispositivo.
Operaciones	Consulta, registro y borrado.

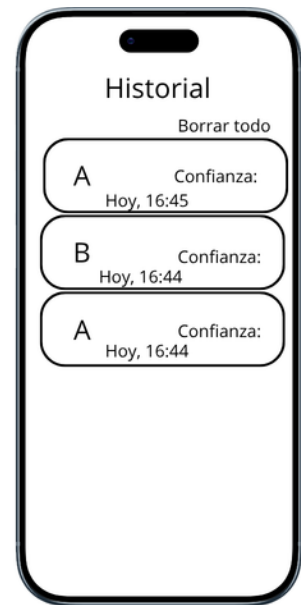


Imagen 4.6.1. Wireframe de pantalla de historial de detecciones

4.7 Pantalla de privacidad

Informa al usuario cómo se maneja la información capturada por la aplicación y qué datos se conservan localmente.

Elemento	Descripción
Funcionalidad	Explicar el uso de datos y la privacidad de la app.
Información	Aviso de privacidad, uso de cámara y tratamiento local del historial.
Tipo de datos	String.
Vigencia del dato	Estático, salvo actualización legal o técnica.
Origen	Texto de la aplicación y política de privacidad.
Operaciones	Consulta.

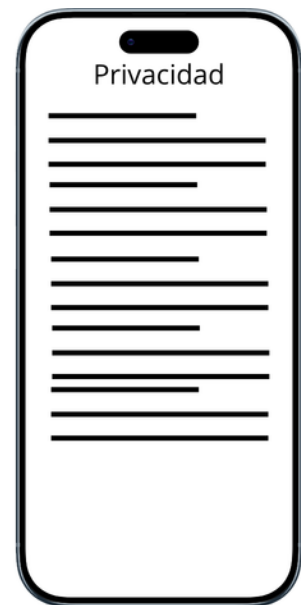


Imagen 4.7.1. Wireframe de pantalla de aviso de privacidad

4.8 Pantalla de detección no identificada

Se presenta cuando no hay mano visible o la confianza de clasificación es baja.

Elemento	Descripción
Funcionalidad	Avisar que la seña no fue reconocida con suficiente certeza.
Información	Estado “No detectada” o “No identificada”.
Tipo de datos	String, Float.
Vigencia del dato	Temporal.
Origen	Procesamiento de Vision y Gemini.
Operaciones	Consulta y actualización.



Imagen 4.8.1. Wireframe de pantalla de detección no identificada

4.9 Pantalla de permisos IA

Solicita autorización para procesar la imagen mediante inteligencia artificial y muestra el consentimiento correspondiente.

Elemento	Descripción
Funcionalidad	Confirmar el uso de IA para analizar la imagen capturada.
Información	Mensaje explicativo y botones de permitir o no permitir.
Tipo de datos	Bool, String.
Vigencia del dato	Temporal, definida por la decisión del usuario.
Origen	Interacción del usuario y configuración interna.
Operaciones	Consulta y registro de consentimiento.



Imagen 4.9.1. Wireframe de pantalla de permisos de IA

4.10 Pantalla de detección analizando letra

Muestra el proceso de análisis cuando la aplicación ya capturó la mano y se encuentra clasificando la letra.

Elemento	Descripción
Funcionalidad	Informar que Vision Framework y Gemini están procesando la letra.
Información	Indicador de carga, texto "Analizando..." y estado del sistema.
Tipo de datos	String, Float.
Vigencia del dato	Temporal, mientras dura el análisis.
Origen	Estado interno de la app.
Operaciones	Consulta y actualización.

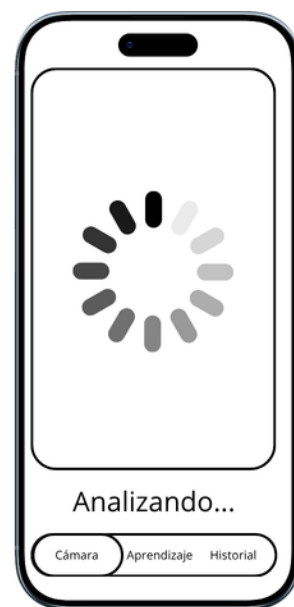


Imagen 4.10.1. Wireframe de pantalla de detección analizando letra

5. Flujo de navegación

Al abrir la app por primera vez, el usuario se encuentra con una pantalla inicial de bienvenida y un tutorial acerca del objetivo de la app y cómo utilizarla y navegar a través de la app, posteriormente, se solicita permiso al usuario para acceder a la cámara, si se concede el permiso, el usuario es dirigido al módulo de cámara, donde el usuario puede iniciar la detección en vivo ya sea utilizando la cámara frontal o trasera para iniciar el análisis de señas en vivo.

Desde la barra de navegación inferior, el usuario puede desplazarse entre tres módulos principales: detección, aprendizaje e historial. Al navegar al módulo de aprendizaje el usuario puede consultar las letras del abecedario junto con una imagen e indicaciones para recrearla, en este mismo módulo el usuario puede pulsar un botón de practicar para dirigirse al módulo de detección para intentar recrear la letra de su elección, a su vez, se puede navegar al módulo de historial donde se muestran las letras detectadas previamente, que han sido traducidas utilizando el módulo de cámara y utilizando deslizamientos puede eliminar elementos del historial.

6. Flujo de datos y cambios de estado

6.1. Flujo de datos

El flujo de datos de la app inicia cuando el usuario activa el módulo de cámara para empezar con la detección en vivo de las letras del alfabeto LSM, la cámara captura frames de video mediante AVFoundation y la app se encarga de recortar la región de interés donde el usuario debe colocar la mano para analizar la imagen, posteriormente Apple Vision Framework analiza la imagen para detectar la postura de la mano y obtener puntos de referencia.

A partir de los puntos obtenidos, la app genera una descripción estructurada de la mano, indicando la orientación de la palma y la orientación y flexión de los dedos para luego verificar si el resultado ya se encuentra en cache, en caso de que no, se envía la descripción geométrica y la imagen al modelo de Gemini para obtener la letra con la mayor probabilidad según los datos recabados.

La respuesta proporcionada por el modelo se transforma en un resultado estructurado que incluye la señal detectada, significado, descripción de la mano, confianza en la detección realizada y fecha y hora de la detección, por último, la app muestra en pantalla el resultado para luego enviar la información a la sección de historial donde se guardan las detecciones con sus datos asociados.

6.2. Cambios de estado

La aplicación presenta una serie de cambios de estado que permiten representar de forma clara el comportamiento del sistema durante la interacción del usuario. Se muestra el siguiente diagrama para representar de manera visual estos cambios de estado.

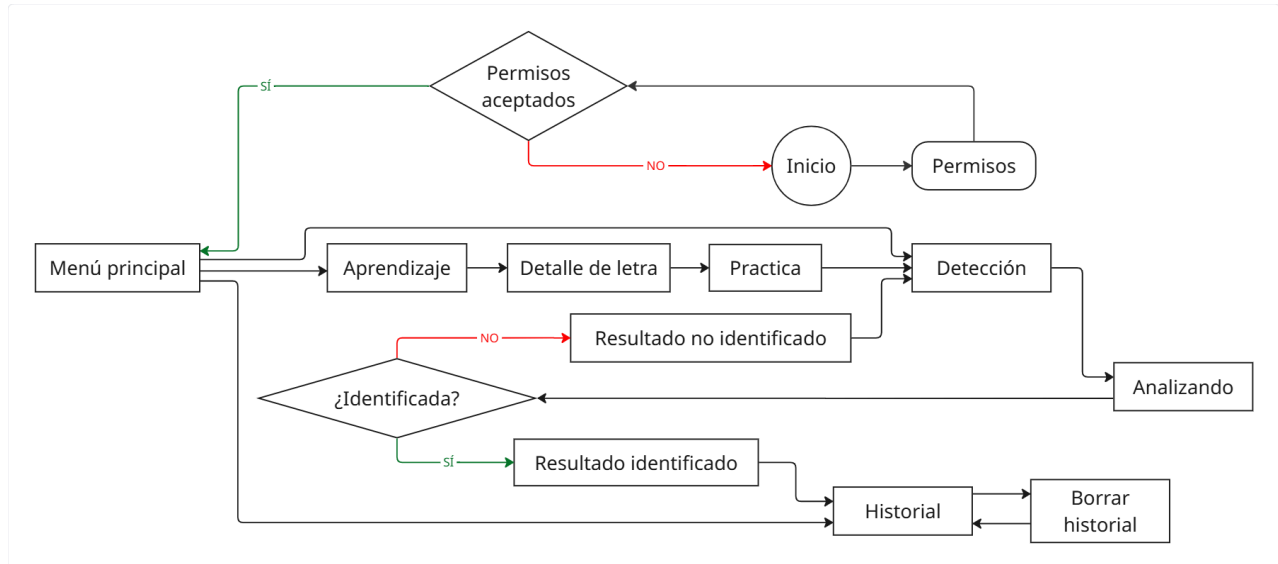


Imagen 6.2.1. Diagrama de cambios de estado.

El flujo principal inicia en el estado de bienvenida, donde la app muestra su propósito y orientación general. Después pasa al estado de solicitud de permisos, en el que iOS pide autorización para acceder a la cámara, requisito indispensable para utilizar el módulo de detección.

Si el usuario concede el permiso, la aplicación entra al estado de captura y análisis, donde la cámara transmite video en tiempo real y Vision Framework obtiene los puntos de referencia de la mano. A partir de estos datos, el sistema genera una descripción geométrica y la envía a Gemini para clasificar la letra detectada. Durante este proceso puede existir un estado intermedio de carga o procesamiento, que informa al usuario que el sistema está analizando la seña en ese momento.

Cuando la detección supera el umbral de confianza establecido, la aplicación cambia al estado de resultado identificado. En este estado se muestra la letra reconocida, su equivalente en el abecedario LSM, el nivel de confianza y, en caso de ser necesario, una retroalimentación visual o háptica. Además, el registro se guarda automáticamente en el historial local del dispositivo.

Si la confianza es baja o no se detecta correctamente la posición de la mano, la aplicación cambia al estado de no identificada. En este caso, se informa al usuario que la

señal no pudo clasificarse con suficiente certeza y se evita guardar un falso positivo en el historial. Este comportamiento ayuda a mantener la calidad de los datos almacenados.

En el módulo de historial existen dos estados adicionales: consulta, cuando el usuario revisa las detecciones guardadas, y eliminación, cuando borra registros de forma individual mediante deslizamiento o elimina todo el historial con confirmación explícita. Por último, el módulo de aprendizaje se mantiene en un estado de consulta estática, ya que solo presenta información de referencia del abecedario y no modifica datos del sistema.

Desde la pantalla de detalle de una letra, el usuario puede regresar al estado de detección mediante el botón “Practicar”, lo que permite repetir el proceso de captura y análisis para reforzar el aprendizaje. Esta transición es importante porque conecta el módulo educativo con la práctica en tiempo real y hace que el flujo de navegación sea cíclico y más útil para el usuario.

7. Dispositivos y sensores

7.1. Dispositivos, tamaños y orientaciones de pantalla

La aplicación está desarrollada para iPhone y iPad con iOS 16 o superior, la interfaz está diseñada principalmente para orientación vertical, ya que este formato facilita el uso de la cámara con una sola mano, sin embargo, la app cuenta con la funcionalidad de interfaz y de cámara para trabajar de forma horizontal sin afectar al rendimiento o la usabilidad de la app.

7.2. Sensores

La app utiliza principalmente ambas cámaras (frontal y trasera) con las que cuentan los dispositivos para los que se desarrolló la app para capturar el video para detectar la mano y las letras LSM, además, se utiliza el sensor de orientación del dispositivo para permitir el cambio de orientación, la app no utiliza otro tipo de sensores como ubicación, micrófono o bluetooth.

8. Demo

Las pantallas incluidas en la demo corresponden a bienvenida, detección, aprendizaje, detalle de letra e historial. Cada una presenta los elementos visuales definidos para la aplicación, como la paleta de colores, la tipografía, la disposición de los componentes y la estructura general de navegación. En conjunto, estas imágenes permiten exponer de forma ordenada la propuesta de diseño y el funcionamiento esperado de la aplicación.

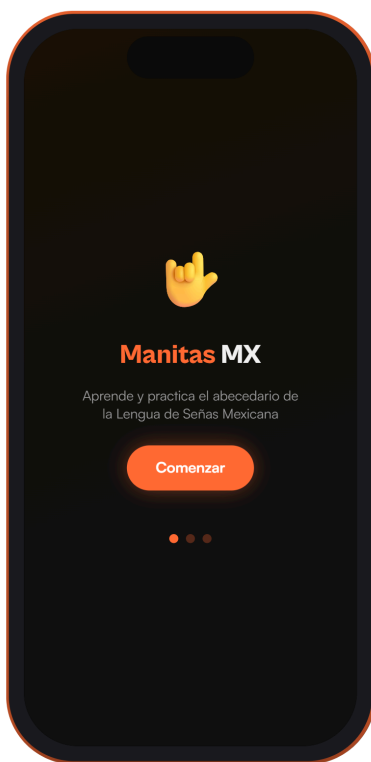


Figura 8.1. Pantalla de bienvenida de Manitas MX

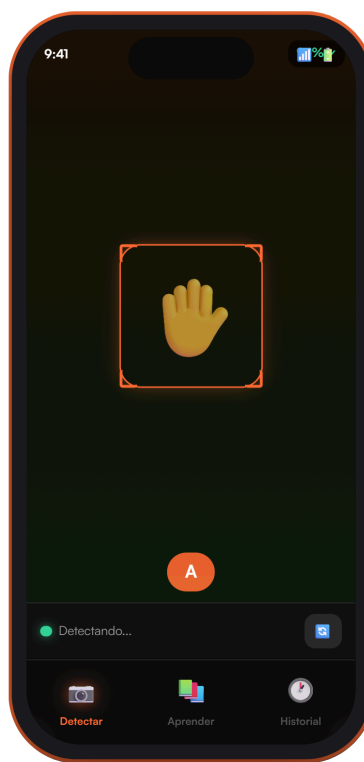


Figura 8.2. Pantalla de detección en tiempo real

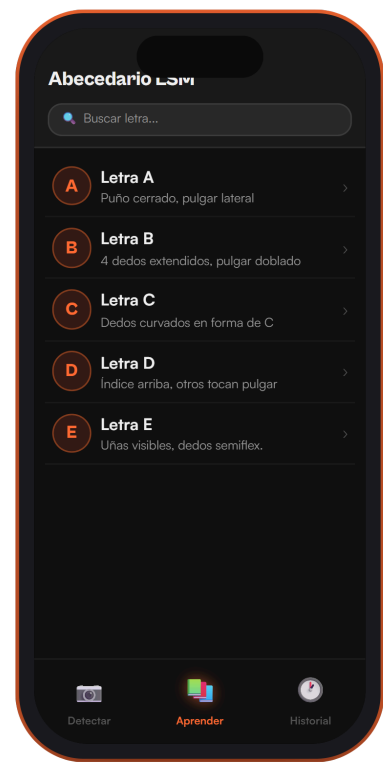


Figura 8.3. Módulo de aprendizaje del abecedario LSM.



Figura 8.4. Módulo de detalle de letra LSM.

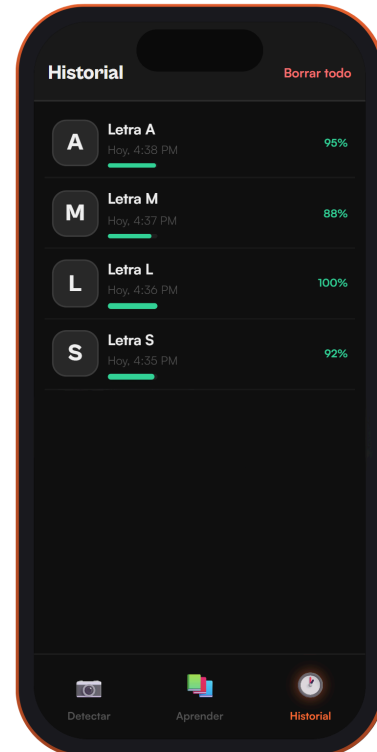


Figura 8.5. Pantalla de historial de detecciones.

9. Lenguajes, herramientas y frameworks

9.1. Lenguajes de programación

Se utilizaron 2 lenguajes de programación para la creación de la app: Swift, que es el lenguaje principal de desarrollo en dispositivos que utilizan iOS y JSON, que fue utilizado para realizar solicitudes y recibir respuestas por parte del modelo de inteligencia artificial de Gemini.

El lenguaje de programación principal utilizado para la creación de la app fue Swift, debido a que es el lenguaje nativo para el desarrollo de aplicaciones para el sistema operativo iOS. Además, se utilizó JSON como formato de intercambio de datos para las diferentes solicitudes para cada letra y con el propósito de procesar respuestas de la API de Gemini.

9.2. Herramientas

Las herramientas utilizadas fueron:

- Xcode: fue utilizado para desarrollar, compilar, ejecutar y probar la aplicación por medio de un dispositivo físico.
- GitHub: para colaborar entre todos los miembros del equipo, controlar las versiones de la app y respaldar el código.
- Google Docs: redacción colaborativa del trabajo de los requerimientos y el reporte final de la app.
- Inteligencia artificial: utilizada durante el desarrollo para la depuración del código, comprensión de errores, ayuda técnica y documentación.

9.3. Frameworks

Los frameworks utilizados para el desarrollo de la app fueron SwiftUI AVFoundation y Vision Framework. SwiftUI se utilizó para construir la interfaz declarativa, manejar vistas, navegación, TabView, estados y componentes reutilizables como botones; se utilizó AVFoundation para configurar la cámara, capturar vídeo y alternar entre la cámara frontal y trasera de acuerdo con la preferencia del usuario y Vision Framework se utilizó para detectar la postura de la mano y extraer puntos de referencia.

10. Arquitectura y metodología

10.1. Arquitectura del proyecto

La arquitectura del proyecto se aproxima a MVVM (Model-View-ViewModel) debido a que SwiftUI maneja vistas declarativas y objetos observables del estado, separando el proyecto en capas: Presentation para las vistas y pantallas, Domain para las reglas y modelos, Data para los datos que se usan en toda la aplicación como el historial e Infrastructure para el manejo de los sistemas como la cámara o la detección de la mano. En la versión MVP de la app, no se cuenta con un backend propio para el manejo de las API keys, es recomendable que para una versión final se agregue dicho backend que controle las cuotas, gestione errores y mejore la seguridad del sistema.

10.2. Patrones de diseño

Para el desarrollo de la app se siguieron diversos patrones de diseño:

- MVVM: separación de vistas, modelos y el estado observable.
- Observer / ObservableObject: actualización de la interfaz cuando existe un cambio de estado.
- Singleton controlado: administración compartida de recursos para la pool de API keys de Gemini y el caché de resultados, es decir, para recursos que se comparten a lo largo de la aplicación.
- Componentización: creación de vistas reutilizables para botones, tarjetas, indicadores y overlays.

10.3. Metodología de desarrollo

La metodología de desarrollo utilizada inicialmente fue de prototipado rápido e iterativo, adecuada para el contexto del hackathon; primero se identificó y definió el problema, el alcance mínimo viable de la app y el público objetivo, posteriormente, se determinaron los módulos que sería necesario desarrollar para que la aplicación fuera funcional (cámara, detección, aprendizaje, historial e interfaz), luego, se construyó una primera versión con las funcionalidades mínimas sobre la que se realizaron iteraciones para mejorar el prompt utilizado para la detección de la letra y ajustarlo de acuerdo con los resultados, a su vez, con cada iteración se corrigieron errores, ajustes de cámara e interfaz y mejoras de optimización y uso de API keys.

10.4. Uso de inteligencia artificial

El uso de la inteligencia artificial se puede separar en dos:

Uso de la inteligencia artificial dentro de la app: Gemini se utilizó para reconocer letras LSM con instrucciones estrictas de salida que posteriormente serían utilizadas para mostrar la traducción en tiempo real.

Uso de la inteligencia artificial como apoyo en el desarrollo: se utilizó para la comprensión de Swift y SwiftUI, estructurar prompts, depurar errores, documentación y ayuda en la toma de ciertas decisiones técnicas.

11. Administración del proyecto

11.1. Organización del proyecto

El proyecto se organizó en varias fases de trabajo: investigación de la problemática, definición del alcance, diseño de interfaz y funcionalidades, desarrollo del prototipo, integración con inteligencia artificial, pruebas técnicas y de usabilidad, documentación y preparación para la entrega final.

11.2. Herramientas de administración

Para la administración y el trabajo del proyecto se utilizaron distintas herramientas: GitHub, para compartir y respaldar código, documentación y entrega final del proyecto, Google Docs para edición colaborativa de la documentación y WhatsApp para la comunicación de todos los miembros del equipo.

11.3. Equipo de trabajo y roles

<i>Integrante</i>	<i>Participación principal</i>	<i>Apoyo complementario</i>
<i>Cruz Buenavista Leslie Sarahí</i>	Redacción y organización de la documentación, integración conceptual de requerimientos.	Revisión general del reporte, apoyo en la presentación y en la definición del alcance.
<i>Martínez Bravo Daniela</i>	Diseño de interfaz, estructuración de pantallas, wireframes y propuesta visual.	Apoyo en la documentación, revisión de pantallas y preparación de la presentación.
<i>Pelaez Semprun Diego Eduardo</i>	Desarrollo técnico, lógica de detección e integración con Gemini.	Apoyo en documentación, presentación y resolución de errores junto con el equipo.

El trabajo del proyecto se realizó de forma colaborativa. En la parte de documentación, cada integrante desarrolló secciones de manera individual para después revisar y unificar el contenido entre todos. En la presentación final, el equipo también se dividió por partes para exponer de forma ordenada. En cambio, en la fase de desarrollo técnico todos participaron juntos, especialmente en la implementación, pruebas y corrección de errores, ya que la resolución de problemas se abordó de manera conjunta.

12. Permisos y publicación de la app

12.1. Permisos requeridos

Para el uso de la app es necesario el permiso para utilizar la cámara para detectar y traducir señales en tiempo real al capturar la mano del usuario, en su versión MVP, la app no requiere acceso a la ubicación, micrófono, contactos, galería de fotos ni Bluetooth, por lo cual no se requiere ningún permiso adicional.

12.2. Adecuaciones necesarias para la publicación

Para la publicación de la app se necesitan realizar ciertas adecuaciones:

- Quitar las API keys del código fuente utilizando un sistema de tokens temporales (servicio que tiene un costo monetario elevado)
- Implementar política de privacidad indicando uso de cámara, envío de imagen a servicio externo y almacenamiento local del historial
- Agregar un manejo de consentimiento para procesamiento de imagen por medio de una inteligencia artificial.

13. Costos y tiempo de desarrollo

13.1. Tiempo estimado de desarrollo

El desarrollo del prototipo inicial realizado para el Hackathon tuvo un tiempo de desarrollo de 2 días, posterior a eso, el ajuste del MVP tuvo un tiempo de desarrollo de 2 semanas, con otras 2 semanas de pruebas técnicas para distintos dispositivos, corrección de errores y mejora del prompt de clasificación por letra, posteriormente se estima un periodo de 3 semanas para pruebas con usuarios reales y asesoría de expertos para el

mejoramiento de la app y por último, la preparación para la publicación de la app, que se estima que tome 1 semana para realizar el aviso de privacidad, materiales para la App Store y revisión de permisos.

Fase	Duración estimada
<i>Prototipo inicial del hackathon</i>	2 días
<i>Ajuste del MVP</i>	2 semanas
<i>Pruebas técnicas y corrección de errores</i>	2 semanas
<i>Pruebas de usuarios reales y expertos</i>	3 semanas
<i>Preparación para la publicación</i>	1 semana
Total	8 semanas y 2 días aproximadamente

13.2. Costos de desarrollo

Para fines académicos, el costo de desarrollo directo se considera bajo, debido a que el proyecto fue realizado con equipo de cómputo del laboratorio y por los miembros del equipo Code&Coffe como parte de la asignatura de Cómputo Móvil, sin embargo, si el proyecto se llevará a cabo en un entorno profesional, deben considerarse costos de desarrollo en iOS, diseño de interfaz, pruebas, documentación y validación con usuarios.

En una implementación profesional, el costo de desarrollo no solo incluye horas de programación, sino también análisis del problema, arquitectura de software, diseño de experiencia de usuario, pruebas de calidad y coordinación del equipo. Por esta razón, aunque el prototipo puede construirse con una inversión reducida, la versión lista para mercado requiere un presupuesto mucho mayor.

13.3. Costos de implementación

Los costos de implementación son los que se necesitan para poner la app en funcionamiento como producto publicable. El gasto más claro es la cuenta de Apple Developer, que se necesita para publicar la app en la App Store. También hay que considerar el uso de Gemini API, cuyo costo depende de cuántos usuarios usen la app y de cuántas veces se haga una detección. Si más adelante se agrega un backend propio, también habría que pagar alojamiento, almacenamiento y monitoreo.

Elemento	Costo estimado
<i>Cuenta Apple Developer</i>	\$99 USD al año
<i>Uso de Gemini API</i>	Variable según número de usuarios y frecuencia de uso
<i>Backend</i>	Variable según infraestructura, almacenamiento y tráfico

13.4. Costos de mantenimiento

Los costos de mantenimiento incluyen renovación de la cuenta Apple Developer, consumo de la API de Gemini, corrección de errores, ajuste por nuevas versiones de iOS, soporte a usuarios y actualizaciones futuras.

Elemento	Costo estimado
<i>Renovación de Apple Developer</i>	\$99 USD al año
<i>Consumo de Gemini API</i>	Pago mensual y de acuerdo con el uso
<i>Corrección de errores</i>	Variable según complejidad y frecuencia de fallos
<i>Actualización por nuevas versiones de iOS</i>	Periódico
<i>Soporte a usuarios</i>	Variable
<i>Actualizaciones futuras</i>	Variable

14. Referencias

[1] Secretaría de Salud. (2021, 28 de noviembre). 530. Con discapacidad auditiva, 2.3 millones de personas: Instituto Nacional de Rehabilitación. Gobierno de México. <https://www.gob.mx/salud/prensa/530-con-discapacidad-auditiva-2-3-millones-de-personas-instituto-nacional-de-rehabilitacion?idiom=es>

[2] Comisión Nacional de Derechos Humanos. (2012, 13 de septiembre). Norma Oficial Mexicana NOM-031-SSA3-2012, Asistencia Social. Prestación de Servicios de Asistencia Social a Adultos y

Adultos Mayores en situación de riesgo y vulnerabilidad. Diario Oficial de la Federación. <https://www.cndh.org.mx/DocTR/2016/JUR/A70/01/JUR-20170331-NOR32.pdf>

[3] Toolify. (s.f.). IA centrada en el ser humano: Guía completa y beneficios. Toolify AI News ES. <https://www.toolify.ai/es/ai-news-es/ia-centrada-en-el-ser-humano-gua-completa-y-beneficios-2591586>

[4] Secretaría de Educación Pública. (2012). Orientaciones para la atención educativa de alumnos sordos que cursan la educación básica, desde el modelo educativo bilingüe-bicultural. Subsecretaría de Educación Básica / Dirección General de Desarrollo Curricular. https://educacionespecial.sep.gob.mx/storage/recursos/2023/05/dTA6GkFrwV-1Libro_Orientacion es.pdf

[5] Ramírez, A. (2025, 7 de marzo). Lengua de señas mexicana. Incluyeme.com. <https://www.incluyeme.com/lengua-de-senas-mexicana/>

[6] Ágora UNIVA. (2023, 5 de septiembre). Lengua de Señas Mexicana, ¿por qué es importante? Universidad del Valle de Atemajac. <https://www.univa.mx/agora/lengua-de-senas-mexicana-por-que-es-importante/>

[7] Jiménez Ortega, M. J. (2024, 28 de octubre). Importante, enseñanza de Lengua de Señas Mexicana. Boletines UAM, (463). <https://boletines.uam.mx/archivos/numero-463/>

[8] Instituto Nacional de Estadística y Geografía. (2017). Encuesta Nacional sobre Discriminación (ENADIS) 2017: Principales resultados. INEGI. https://www.inegi.org.mx/contenidos/programas/enadis/2017/doc/enadis2017_resultados.pdf

[9] Consejo Nacional para Prevenir la Discriminación. (2023). Ficha temática: Discriminación en contra de las personas con discapacidad. CONAPRED. https://conapred.gob.mx/wp-content/uploads/2023/11/FT_PDDiscapacidad_2023.pdf

[10] Ley General para la Inclusión de las Personas con Discapacidad, Diario Oficial de la Federación [DOF], 30 de mayo de 2011 (México), última reforma 14 de junio de 2024. <https://www.diputados.gob.mx/LeyesBiblio/pdf/LGIPD.pdf>

[11] Microsoft. (2023). AI for Accessibility Impact Report 2023. Microsoft Corporation. <https://www.microsoft.com/en-us/ai/ai-for-accessibility>

[12] Modelo Anáhuac Querétaro de Naciones Unidas. (2024). Reformas educativas para la inclusión de la comunidad sorda en instituciones educativas en América Latina [Guía de fondo, Comité UNICEF, AQMUN 2024]. Universidad Anáhuac Querétaro. https://anahuacqro.edu.mx/aqmun/wp-content/uploads/2024/01/UNICEF_02.pdf